

How Lumeria Works

A map of the Streamlit app that replays Luméria game sessions: where data comes from, how it flows through the codebase, and what's worth studying in it.

streamlit_app/ · 20 files covered · video sync included here — excluded only from the code review docs

[2-minute version](#)[5-minute version](#)[System architecture](#)[Session load & render flow](#)[Video sync: two export methods](#)[Learning notes](#)[What to study next](#)

IF YOU HAVE TWO MINUTES

The elevator version

Lumeria is a research tool for reviewing a wayfinding game. A student plays Luméria in a virtual medieval city, and this app replays that session — where they walked, which attempts they made, what events happened — as an interactive map.

Data comes from one of two places: a JSON save file (either uploaded directly, or fetched from Firestore where the game already pushed it). The app parses that file into a handful of pandas DataFrames, figures out which level the session was played on, and pulls in that level's map image and building layout to match. Everything downstream — the static map, the animated replay, the CSV/PNG/MP4 exports — reads from that one parsed bundle, cached once per session so the expensive parsing doesn't re-run on every click.

The codebase itself is split into three honest layers: `views/` (the pages you click through), `components/` (the sidebar and download buttons those pages share), and `modules/` (plain Python — parsing, figure-building, no Streamlit inside). That split is the one architectural rule worth remembering; almost everything else follows from keeping it.

The fuller picture

Where a session comes from

Two entry points, one destination:

- **Upload** — the sidebar's file uploader saves the JSON to a temp path, `data_loader.get_game_data_dict()` opens and parses it.
- **Firestore** — the "Firestore Test" page fetches a session document and hands its `gameData` field straight to `get_game_data_dict_from_dict()` — the same parser, so both paths can't drift apart.

Either way, parsing raises a typed `GameDataError` on malformed input instead of crashing — caught by the page, shown as `st.error()`.

What "level-aware" means here

Every session records a `level_number`. That one field now drives three separate lookups: the map's grid size (`LEVELS`), its background image (`MAP_IMAGES`), and which rows of the one shared `buildings.json` belong to this level. All three live behind a single function, `get_loaded_data()` — get the level wrong once, everything downstream is wrong consistently, which is exactly what you want.

Caching, the Streamlit way

Streamlit re-runs the whole script top-to-bottom on every click.

`st.session_state` is the one thing that survives a rerun, so the sidebar loads a session once and stashes it under `st.session_state["loaded_data"]`. Every page after that just reads it back — no reload, no re-parse — until something invalidates it (a new upload, a new Firestore session, "Reset to example data").

From parsed data to pixels

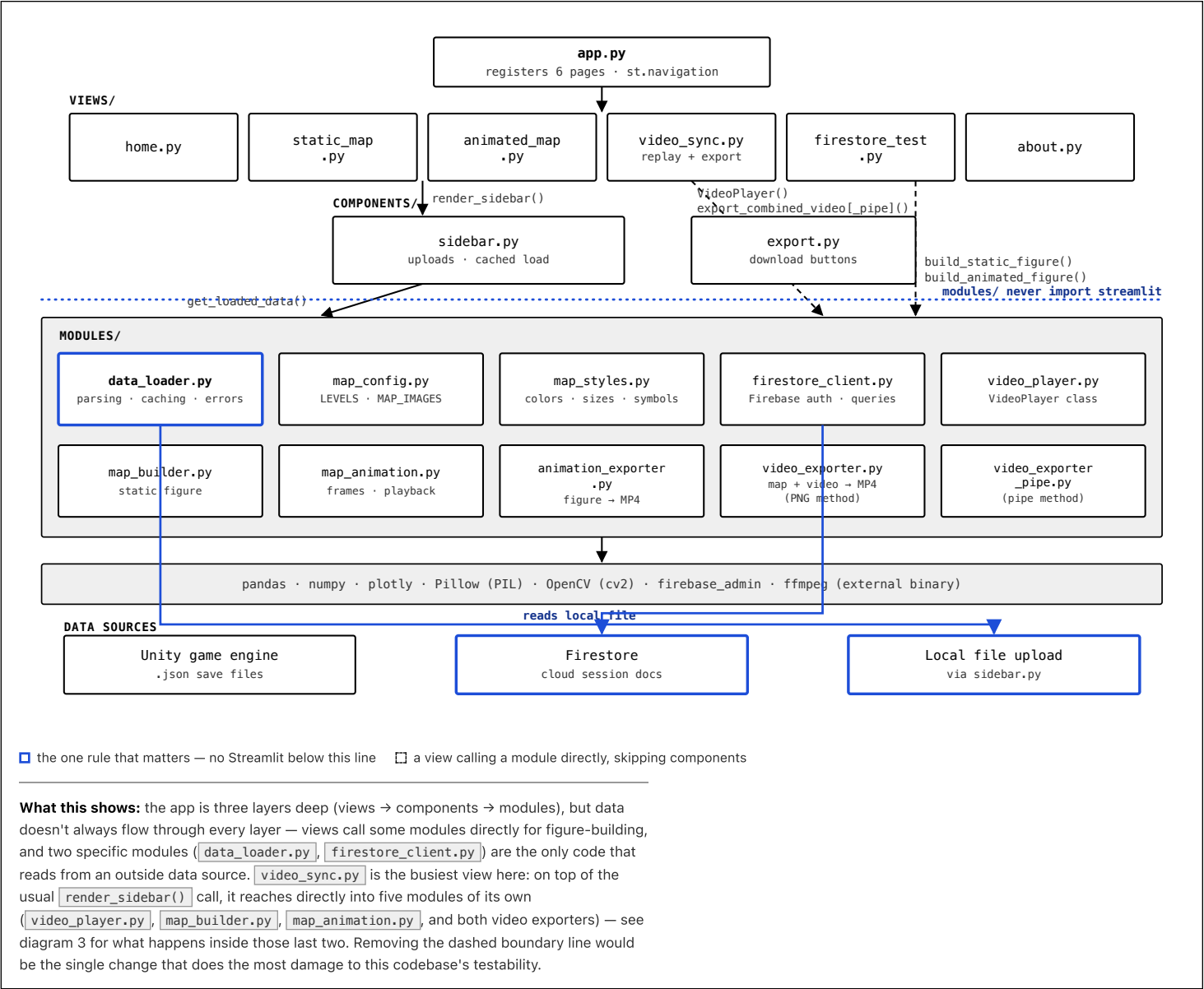
`map_builder.py` draws the static picture: background image, POI markers, trails, attempt paths, game events, all as Plotly traces on one figure. `map_animation.py` takes that same base figure and adds one animation frame per recorded movement sample, plus play/pause/speed controls. Both read their visual constants — colors, sizes, symbols — from one place, `map_styles.py`, so a palette change updates every page at once.

The video sync page — a live A/B test

`video_sync.py` lines up the animated replay next to the actual gameplay recording, frame for frame, using the same `get_nearest_index_for_time()` lookup as everywhere else. It's also the one page still mid-refactor: it ships two ways to export the combined video — `video_exporter.py` (writes PNGs to disk, then one `ffmpeg` call) and `video_exporter_pipe.py` (streams frames straight to `ffmpeg`'s stdin) — kept side by side on purpose, to compare speed and file size before picking one. See diagram 3 below.

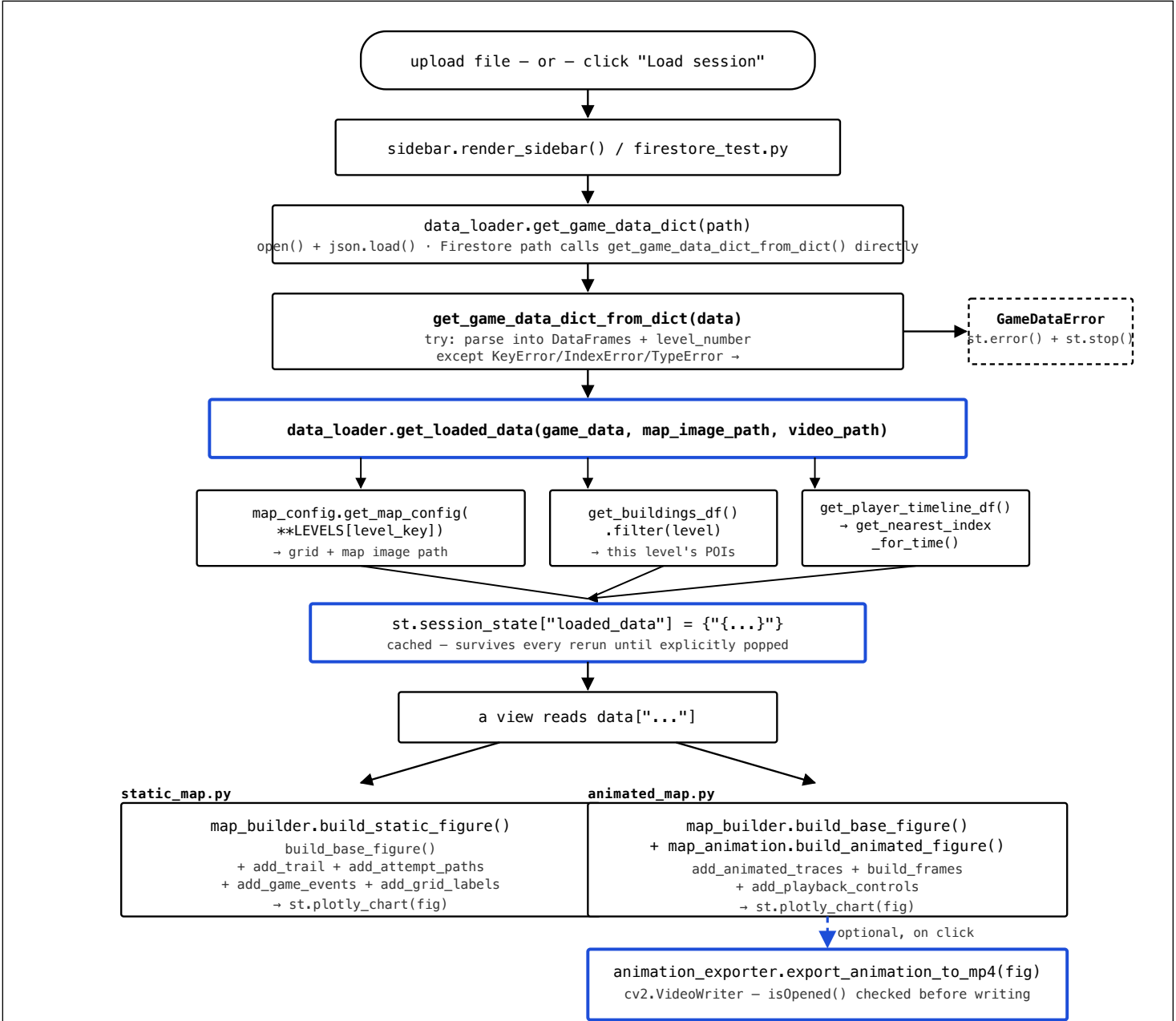
System architecture

How the three code layers stack, and where the two external data sources actually enter — not through the top of the stack, but by being read directly by specific modules.



Loading & rendering a session

The one mechanism worth tracing in full: what actually happens, function by function, between "a file lands in the sidebar" and "a chart appears on screen" — including the one branch where it can fail.

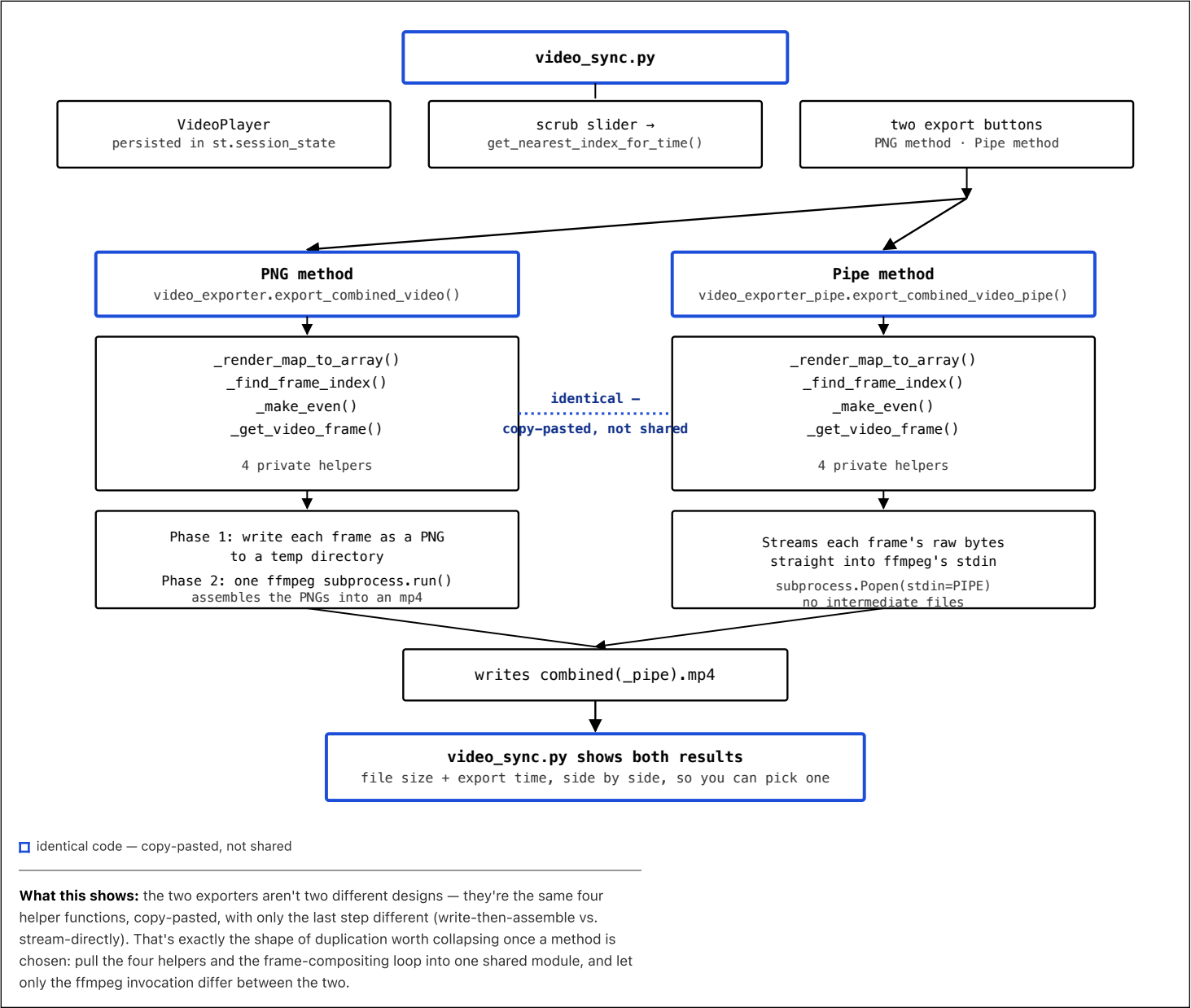


the single-source-of-truth call — every view's data passes through here branch or optional path — not the main line

What this shows: everything funnels through `get_loaded_data()` exactly once per session, which is why fixing the level-number bug there fixed it for both the upload path and the Firestore path at the same time. Below that point, the diagram forks — static and animated pages call different `map_builder`/`map_animation` functions on the same cached data, which is the whole reason a static map and an animated map never disagree about where the player actually walked.

Video sync: two export methods, one duplicated core

The one place in this app that's deliberately unfinished: two files that do almost exactly the same thing, kept side by side so their speed and output size can be compared before one gets deleted.



Ideas worth sitting with

Seven things this codebase actually demonstrates — not abstractly, but because you ran into each one directly while fixing something.

01 Streamlit's rerun model is the reason `session_state` exists

Every click — a button, a slider, a file upload — re-runs the *entire script* from the top. Normal variables get wiped and rebuilt each time; nothing survives a rerun except what you explicitly put in `st.session_state`. That single fact explains almost every design decision in `sidebar.py` and `video_sync.py`: the `if "loaded_data" not in st.session_state:` guards are manual memoization, and the `.pop("loaded_data", None)` calls scattered through the upload handlers are manual cache invalidation. There's no magic here — it's the same cache-and-invalidate pattern you'd write anywhere, just spelled out by hand because Streamlit doesn't do it for you automatically.

02 Duplicated logic doesn't stay identical — it drifts

Before `get_loaded_data()` existed, `sidebar.py` and `firestore_test.py` each built the same session dict independently. When level-awareness was added, it landed in one copy and not the other — so a Firestore session on a non-default level would silently get the wrong map, with no error. That wasn't a typo bug; it was a structural one. The fix wasn't "be more careful next time," it was removing the second copy entirely. **Two implementations of the same idea are not twice as safe — they're a standing bet that nobody ever changes just one of them.**

03 Typed exceptions at the boundary, not everywhere

`GameDataError` and `FirestoreConnectionError` both follow the same shape: a pure module raises one specific exception type when its contract is violated (bad JSON, bad credentials), and the view layer is the only place that catches it and turns it into `st.error()`. Notice what *isn't* wrapped in try/except: the well-tested pandas transformations in the middle of `get_game_data_dict_from_dict()` aren't individually guarded — one broad `try` around the whole parse is enough, because every failure there resolves to the same useful message. Catch at the boundary where you can actually do something about it (show a message), not at every line where something could theoretically go wrong.

04 The pandas idiom worth stealing: `idxmax()` for "last one before X"

`get_nearest_index_for_time()` is four lines that do a lot: `df[df[time_col] < time][time_col]` filters to every row before the target time — vectorized, no Python loop — then `.idxmax()` finds the index of the largest (i.e. most recent) one among them. That's the whole "what was true at this moment" pattern used everywhere in this app: which attempt was active, where the player was standing, which frame to show. It's worth understanding this one function cold, because three other functions are thin wrappers around it.

05 Boundary conditions are a decision, not a detail

When the video-sync slider was refactored to reuse `get_nearest_index_for_time()`, the real question wasn't "does this compile" — it was: what happens when the slider sits *exactly* on a recorded timestamp? Strict `<` means that sample belongs to whatever came before it; `<=` means it belongs to itself. Both are defensible, but they give different answers, and the "recording happens every second, we want the last known state before this time" reasoning is what actually settled it — not a coin flip. The lesson: when you hit a boundary case, trace what each option actually does before picking one. "It works on my test file" isn't the same as "I know what happens at the edge."

06 A comment that's wrong is worse than no comment

`map_builder.py`'s `add_grid_labels()` has a comment — `"# One fewer cell than grid points in each direction"` — that doesn't describe what the code below it does anymore; it's a leftover from an earlier version of the logic. A missing comment costs a reader a few seconds of re-deriving intent from the code. A *wrong* comment costs more: it actively points a future reader (including future-you) toward a false explanation, and they'll trust it before they trust the code. Comments explain *why*, and that "why" needs the same maintenance discipline as the code it's attached to — when the logic changes, the comment either gets updated or deleted, never left behind.

07 Not all duplication is a mistake — some of it is a live experiment

`video_exporter.py` and `video_exporter_pipe.py` (diagram 3) look exactly like the kind of duplication note 02 warns about — four identical helper functions, copy-pasted. The difference is that this one is intentional: it exists specifically so the two export strategies can be timed and measured against each other on real data before either gets deleted. That's a legitimate reason to tolerate duplication temporarily. The tell that it's time to collapse it isn't "duplication is bad" in the abstract — it's a decision having actually been made. Once you know which method you're keeping, the unchosen file and the duplicated helpers in the chosen one both stop earning their keep, and note 02's warning applies again in full.

What to study next, in this codebase specifically

You're already adding type hints as you go — here's what pairs well with that pass.

TYPE HINTS

Fix

data_loader.py

's

level_number:
str

hint first — it's wrong, not missing
(the real value is an

int

). A wrong hint is worse to inherit than no hint, for the
same reason as note 06 above.

DATACLASSES

MapBounds

in

map_config.py

is

loaded_data

dict that

@dataclass

would give

data["..."]

lookup

KeyError

ty

already

a small,

clean

example.

The

flows

through

this whole

app —

seven

keys, built

in exactly

one place

— is a

strong

candidate

for the

same

treatment:

a

every

downstream

autocomplete

and a real

type error

instead of a

silent

TESTING

data_loader.py

's pure

functions

(

get_nearest_index_for_time

,

get_challenge_value_for_time

,

get_player_timeline_df

) take plain

pytes

DataFrames

in and

return plain

values out

— no

Streamlit,

no file I/O,

no mocking

required.

They're the

cheapest,

highest-

value place

in this

codebase

to write

your first

real

ARCHITECTURE

When you're ready to pick the

video-sync cluster back up, the

two exporters (

video_exporter.py

,

video_exporter_pipe.py

) are ~90% identical — a good exercise in extracting a

shared module once you've decided which export

method to keep.